

Báo cáo Nghiên cứu Chuyên sâu: Bản chất Hoạt động, Kiến trúc và Giới hạn Quy mô của CloudSim Plus

Giới thiệu Tổng quan về Nền tảng Mô phỏng Điện toán Đám mây

Trong bối cảnh kỷ nguyên số, điện toán đám mây (Cloud Computing) đã khẳng định vị thế là một hạ tầng cốt lõi cho việc triển khai các ứng dụng phân tán, cung cấp tài nguyên máy tính có khả năng mở rộng linh hoạt theo nhu cầu. Việc nghiên cứu, phát triển và tối ưu hóa các thuật toán quản lý tài nguyên, định tuyến máy ảo (Virtual Machine - VM), quản lý năng lượng và đáp ứng Thỏa thuận mức dịch vụ (SLA) đòi hỏi các môi trường thử nghiệm phức tạp. Tuy nhiên, việc triển khai các kịch bản thử nghiệm quy mô lớn trên các trung tâm dữ liệu (Datacenter) thực tế như Amazon Web Services hay Google Cloud là vô cùng đắt đỏ, tốn thời gian và gần như không thể tái lập (reproducibility) một cách nhất quán do các điều kiện môi trường liên tục biến động.¹

Để giải quyết bài toán này, các nền tảng mô phỏng trên máy tính đã ra đời, trong đó CloudSim là một trong những công cụ được sử dụng rộng rãi nhất trong giới học thuật và công nghiệp.¹ Mặc dù vậy, phiên bản CloudSim nguyên bản tồn tại nhiều hạn chế về thiết kế kiến trúc, chứa lượng lớn mã nguồn lặp lại, thiếu các bài kiểm thử tích hợp (integration tests) và vi phạm nhiều nguyên tắc thiết kế hướng đối tượng (Object-Oriented Design).¹ Nhằm khắc phục triệt để các rào cản kỹ thuật này, CloudSim Plus đã được phát triển như một dự án phân nhánh (fork) độc lập, tái thiết kế toàn diện dựa trên ngôn ngữ Java hiện đại (hỗ trợ các phiên bản JDK 17, 21 và 24+).⁴

Báo cáo này cung cấp một phân tích toàn diện và sâu sắc về bản chất hoạt động của CloudSim Plus, đi sâu vào cấu trúc xử lý sự kiện, cơ chế mô phỏng máy chủ vật lý, các giới hạn phần mềm, và đặc biệt là phân tích các điểm nghẽn (bottlenecks) khi mở rộng quy mô hệ thống lên ngưỡng hàng vạn máy chủ.

Kiến trúc Hệ thống và Triết lý Kỹ thuật Phần mềm

Sự khác biệt nền tảng giữa CloudSim và CloudSim Plus nằm ở việc áp dụng nghiêm ngặt các nguyên tắc kỹ thuật phần mềm (Software Engineering), đặc biệt là nguyên tắc Tách biệt Mối quan tâm (Separation of Concerns - SoC) và các nguyên lý SOLID.¹ Nền tảng này được thiết kế để cung cấp một bộ công cụ có tính mô-đun cao, dễ hiểu, dễ duy trì và mở rộng.

Kiến trúc của dự án được quản lý bởi công cụ Maven, chia thành các phân hệ (modules) độc lập nhưng tương tác chặt chẽ với nhau, giúp các nhà nghiên cứu dễ dàng tích hợp và phát triển các lớp tùy chỉnh mà không cần can thiệp vào mã nguồn lõi.⁶ Việc tái cấu trúc hệ thống này cho

phép mã nguồn giảm thiểu sự dư thừa, đồng thời tăng cường độ an toàn của dữ liệu thông qua cơ chế kiểm tra kiểu tĩnh (static type checking) của Java.

Phân hệ (Module)	Chức năng Cốt lõi và Vai trò trong Hệ sinh thái
cloudsimplus	Phân hệ lõi (Core API) chứa toàn bộ các giao diện, lớp trừu tượng và logic mô phỏng sự kiện rời rạc. Đây là phân hệ duy nhất bắt buộc phải có để khởi tạo bất kỳ kịch bản mô phỏng nào. Nó bao gồm các gói xử lý mạng, cấp phát tài nguyên, và thuật toán lập lịch. ⁶
cloudsimplus-examples	Tập hợp phong phú các kịch bản mô phỏng mẫu có thể thực thi ngay lập tức. Các ví dụ trải dài từ việc khởi tạo một Datacenter cơ bản đến các kịch bản phức tạp như di dời VM liên đám mây (inter-datacenter VM migration), tối ưu hóa điện năng và đọc tệp dấu vết (trace files) từ Google Cluster Data. ⁶
cloudsimplus-testbeds	Phân hệ hỗ trợ thiết kế các môi trường thử nghiệm lặp lại (repeatable testbeds). Nó cho phép nhà nghiên cứu thực thi tự động nhiều vòng lặp mô phỏng cho cùng một tham số để tính toán các giá trị trung bình, độ lệch chuẩn và khoảng tin cậy theo tiêu chuẩn nghiên cứu khoa học. ⁶
cloudsimplus-benchmarks	Phân hệ chuyên dụng sử dụng nội bộ để đo lường độ trễ (overhead) và đánh giá hiệu năng phần cứng/phần mềm của chính các tính năng bên trong framework. Nó giúp các nhà phát triển đảm bảo rằng các bản cập nhật mới không làm suy giảm tốc độ xử lý của hệ thống. ⁹

Triết lý thiết kế hướng đối tượng mạnh mẽ (Strongly Object-Oriented Framework) của CloudSim Plus thể hiện rõ qua việc loại bỏ cơ chế truyền thông điệp bằng các mảng số nguyên (integer arrays) dễ gây lỗi của phiên bản trước. Thay vì lưu trữ định danh của các thực thể dưới dạng các biến nguyên thủy và phải tìm kiếm chéo thông qua các bảng băm (HashMaps) toàn

cục, CloudSim Plus đóng gói trực tiếp các tham chiếu đối tượng.¹¹ Chẳng hạn, một tác vụ ảo (Cloudlet) chứa trực tiếp tham chiếu đến máy ảo (VM) đang thực thi nó, máy ảo chứa tham chiếu đến máy chủ vật lý (Host), và máy chủ chứa tham chiếu đến Datacenter. Kiến trúc này cho phép gọi các phương thức theo chuỗi liên kết một cách an toàn và trực quan mà không vấp phải các ngoại lệ con trỏ rỗng (NullPointerException).⁶

Cơ chế Xử lý Sự kiện Rời rạc (Discrete Event Simulation - DES)

Bản chất cốt lõi của CloudSim Plus là một hệ thống Mô phỏng Sự kiện Rời rạc (Discrete Event Simulation - DES). Khác với các hệ thống mô phỏng thời gian liên tục (continuous simulation) tính toán lại trạng thái ở từng phân số nhỏ của giây, hệ thống DES chỉ thực sự thay đổi trạng thái và tiến triển thời gian khi một "sự kiện" (event) xác định xảy ra, giúp tăng tốc độ xử lý lên nhiều lần.¹²

Cấu trúc Hàng đợi và Lỗi Xử lý Sự kiện

Động cơ mô phỏng trung tâm được bao bọc bên trong lớp CloudSim (hoặc CloudSimPlus), đóng vai trò như một nhạc trưởng điều phối toàn bộ vòng đời của hệ thống. Đồng hồ mô phỏng bên trong hệ thống (simulation clock) không trôi qua theo thời gian thực (wall-clock time) mà nhảy vọt (advance) từ sự kiện này sang sự kiện khác dựa trên dấu thời gian (timestamp) của chúng.¹² Sự vận hành của DES phụ thuộc hoàn toàn vào hai cấu trúc dữ liệu hàng đợi cốt lõi:

Hàng đợi Tương lai (FutureQueue) là nơi lưu trữ tất cả các sự kiện đã được lên lịch nhưng chưa đến thời điểm thực thi. Hàng đợi này được cấu trúc dưới dạng một Hàng đợi Ưu tiên (Priority Queue), trong đó các sự kiện được sắp xếp một cách tự động theo trình tự thời gian xảy ra.¹³ Mỗi khi một thực thể muốn tương tác với hệ thống trong tương lai (ví dụ: một VM dự kiến sẽ hoàn thành tác vụ sau 10 giây), nó sẽ tạo ra một đối tượng SimEvent chứa định danh loại sự kiện (được định nghĩa trong lớp CloudSimTags), tham chiếu đến thực thể gửi, thực thể nhận, độ trễ và tải trọng dữ liệu đi kèm.¹²

Khi đồng hồ mô phỏng chạm đến mốc thời gian của sự kiện đầu tiên trong FutureQueue, hệ thống sẽ lấy sự kiện đó ra và chuyển vào Hàng đợi Trì hoãn (DeferredQueue). DeferredQueue hoạt động như một vùng đệm (buffer) chứa các sự kiện sẵn sàng để xử lý, hoặc các sự kiện đang bị tạm dừng chờ tài nguyên giải phóng.¹⁴

Sự Dịch chuyển từ Cơ chế Đa luồng (Threads) sang Đối tượng (Java Objects)

Một trong những khía cạnh kỹ thuật quan trọng nhất cần làm rõ khi phân tích CloudSim Plus là cách thức hệ thống biểu diễn và thực thi các thực thể (như Datacenter, Broker, Host, VM). Sự chuyển đổi từ mô hình Đa luồng (Multi-threading) sang mô hình Đối tượng đơn thuần (Plain Old Java Objects) là chìa khóa giải quyết bài toán khả năng mở rộng.

Trong các nguyên mẫu sơ khai và các công cụ mô phỏng phân tán đời đầu (dựa trên bộ máy

SimJava), mỗi một thực thể mô phỏng (entity) được thiết kế là một lớp kế thừa từ giao diện Runnable và được cấp phát một Luồng (Java Thread) riêng biệt trên hệ điều hành.¹⁶ Các Thread này mô phỏng sự chạy song song của các trung tâm dữ liệu và trao đổi dữ liệu với nhau thông qua hàng đợi sự kiện dùng chung. Tuy nhiên, kiến trúc Thread-per-Entity này là một thảm họa về khả năng mở rộng (scalability bottleneck).¹⁶

Khi quy mô mô phỏng tăng lên hàng vạn Host và hàng trăm ngàn VM, việc sinh ra hàng trăm ngàn Threads là điều bất khả thi đối với bất kỳ máy ảo Java (JVM) hay hệ điều hành nào. Hàng trăm ngàn Threads này sẽ tiêu tốn một lượng khổng lồ bộ nhớ RAM chỉ để duy trì các ngăn xếp (Thread Stack), đồng thời tạo ra chi phí chuyển đổi ngữ cảnh (Context Switching) và cạnh tranh tranh chấp khóa (Lock Contention) quá lớn, đánh sập năng lực tính toán của CPU.¹⁶

Để triệt tiêu điểm nghẽn này, CloudSim Plus đã thiết kế lại toàn bộ hệ thống theo mô hình hướng đối tượng điều khiển bằng sự kiện (Event-driven Object Model). Tất cả các thực thể mô phỏng hiện đại, kế thừa từ lớp trừu tượng CloudSimEntity, thực chất **chỉ là các đối tượng Java (Java Objects) thông thường cư trú trên không gian Heap của JVM, hoàn toàn không phải là Thread hay Process.**¹²

Vòng lặp mô phỏng trung tâm chạy trên một **Luồng thực thi duy nhất (Single-threaded Execution Loop)**. Luồng duy nhất này sẽ liên tục rút các đối tượng SimEvent từ FutureQueue, tự động định tuyến đến đối tượng CloudSimEntity đích, và gọi trực tiếp phương thức processEvent(SimEvent) của đối tượng đó.²⁰ Cơ chế này mang lại ba lợi ích tối hậu:

1. **Loại bỏ chi phí chuyển đổi ngữ cảnh:** Lỗi mô phỏng không cần nhờ hệ điều hành cấp phát hay luân chuyển tài nguyên CPU giữa các Thread, mọi thứ diễn ra ở cấp độ tham chiếu đối tượng trong bộ nhớ, tối đa hóa thông lượng chỉ lệnh.
2. **Đảm bảo tính xác định (Determinism):** Do không có sự cạnh tranh ngẫu nhiên giữa các Thread (race conditions), kết quả mô phỏng luôn lặp lại y hệt qua vô số lần chạy nếu giữ nguyên tham số đầu vào, đảm bảo tính chặt chẽ trong nghiên cứu khoa học.²²
3. **Tiết kiệm bộ nhớ:** Hàng triệu đối tượng Java có thể được tạo ra và quản lý bởi bộ thu gom rác (Garbage Collector) một cách trơn tru với chi phí theo dõi bộ nhớ cực thấp so với Thread hệ điều hành.⁵

Mặc dù lõi hệ thống là đơn luồng, CloudSim Plus vẫn cung cấp cơ chế Chạy Mô phỏng Song song (Parallel Execution) ở cấp độ vĩ mô. Nhà nghiên cứu có thể khởi chạy nhiều kịch bản mô phỏng độc lập cùng một lúc (mỗi kịch bản có một luồng lõi riêng) trên các hệ thống CPU đa nhân để rút ngắn thời gian thử nghiệm cho các nghiên cứu Monte Carlo.⁶

Thực thi Đồng bộ và Tương tác Thời gian thực

Để vượt qua giới hạn của việc phải chờ hệ thống mô phỏng chạy xong toàn bộ vòng đời mới thu thập được dữ liệu, CloudSim Plus giới thiệu tính năng Thực thi Đồng bộ (Synchronous Simulation). Thông qua phương thức simulation.startSync(), vòng lặp mô phỏng không tự động tiêu thụ toàn bộ hàng đợi mà trao quyền kiểm soát cho người dùng.

Trong cấu trúc này, nhà nghiên cứu có thể sử dụng vòng lặp while kết hợp với phương thức `simulation.runFor(INTERVAL)` để cho phép hệ thống chỉ mô phỏng tiến lên một khoảng thời gian nhất định (ví dụ: 10 giây mô phỏng), sau đó tạm dừng. Tại điểm dừng này, các kỹ sư có thể trích xuất các số liệu thống kê (metrics) hiện tại, thu thập trạng thái sử dụng tài nguyên (CPU, RAM), và tự động bơm thêm (inject) các máy ảo mới, tạo ra các sự kiện hủy diệt VM, hoặc áp dụng các thuật toán Học Tăng cường (Reinforcement Learning) dựa trên phần thưởng của môi trường, rồi mới cho phép mô phỏng tiếp tục.²⁴ Tính năng này mở ra một kỷ nguyên mới cho việc tích hợp mô phỏng với trí tuệ nhân tạo (AI).

Kiến trúc Mô phỏng Máy chủ (Host) và Quản lý Tài nguyên

Máy chủ vật lý (Host) là thành tố cơ sở cung cấp năng lực tính toán trong môi trường điện toán đám mây. Tại CloudSim Plus, một Host không chỉ là một cấu trúc dữ liệu tĩnh mà là một tập hợp các thành phần điều phối tương tác trực tiếp với nhau theo thời gian thực nhằm mô phỏng các giới hạn phần cứng khắt khe.

Định nghĩa và Khởi tạo Thông số Phần cứng

Hệ thống biểu diễn máy chủ thông qua lớp `HostSimple` hoặc `HostDynamic` (cho phép động học tài nguyên).²⁵ Mỗi Host được đặc trưng bởi bộ thông số cốt lõi:

- **RAM (Megabytes):** Bộ nhớ chính của hệ thống.
- **Storage (Megabytes):** Dung lượng lưu trữ khả dụng cho việc chứa ảnh đĩa (VM images) và dữ liệu.
- **Bandwidth (Megabits/s):** Băng thông mạng nội bộ phân bổ cho giao tiếp dữ liệu.
- **Processing Elements (PEs):** Danh sách các lõi vi xử lý vật lý. Khả năng tính toán của mỗi lõi được đo bằng MIPS (Million Instructions Per Second).¹

Để quản lý các tài nguyên này không bị vượt quá giới hạn phần cứng khi cấp phát cho nhiều VM, CloudSim Plus thiết kế các giao diện `ResourceProvisioner` cho từng loại tài nguyên độc lập. Khi một VM được đưa vào Host, `RamProvisioner` và `BwProvisioner` sẽ kiểm tra xem liệu tài nguyên còn lại có đáp ứng đủ yêu cầu (Demand Vector) hay không. Nếu không đủ, yêu cầu khởi tạo VM sẽ thất bại. Tuy nhiên, hệ thống hỗ trợ khái niệm Cấp phát Vượt mức (Oversubscription), mô phỏng kỹ thuật Phân trang Bộ nhớ ảo (Virtual Memory Paging) của các Hypervisor hiện đại (như VMware ESXi hay KVM). Khi đó, tổng RAM yêu cầu của các VM có thể lớn hơn RAM vật lý, dẫn đến sự suy giảm hiệu năng theo công thức penalty được định nghĩa sẵn.⁶

Lớp Quản lý Tài nguyên	Vai trò Mô phỏng và Cách thức Hoạt động
------------------------	---

PeProvisionerSimple	Quản lý vòng đời và khả năng cấp phát của từng lõi CPU vật lý (PE). Nó theo dõi tổng số MIPS đã phân bổ cho các VM và đảm bảo không vượt quá công suất tối đa của lõi. ⁷
RamProvisionerSimple	Quản lý bộ nhớ RAM tĩnh của Host. Hỗ trợ từ chối cấp phát hoặc chia sẻ giới hạn tùy thuộc vào mô hình oversubscription. ²⁷
BwProvisionerSimple	Quản lý băng thông mạng của Host, điều tiết luồng dữ liệu vào/ra cho các ứng dụng chạy trên VM. ²⁷
VmScheduler	Bộ não điều phối CPU. Đóng vai trò như bộ lập lịch của hệ điều hành máy chủ vật lý, quyết định VM nào được chạy trên lõi PE nào ở mỗi mốc thời gian. ⁷

Bộ Lập lịch Máy ảo (VmScheduler) và Phân chia CPU

Quy trình quản lý các yêu cầu điện toán của VM đối với lõi CPU (PE) vật lý là khía cạnh phức tạp nhất của mô phỏng. Khi một Host nhận các VM, VmScheduler sẽ tính toán cách chia nhỏ công suất MIPS của Host. Có ba mô hình lập lịch VM cốt lõi trong hệ thống:

- **VmSchedulerSpaceShared (Phân bổ Không gian Mở rộng):** Đây là cơ chế cấp phát tĩnh và không ưu tiên (non-preemptive). Trong cơ chế này, các lõi PE vật lý được ánh xạ trực tiếp và độc quyền cho các vCPU của VM. Nếu một Host có 8 lõi và một VM cần 4 lõi, 4 lõi đó sẽ bị khóa hoàn toàn. Bất kỳ VM nào đến sau nếu yêu cầu số lõi lớn hơn số lượng chưa cấp phát sẽ ngay lập tức bị từ chối, bất kể các VM đang chạy có thực sự dùng hết MIPS hay không. Mô phỏng này sát với thực tế của các loại máy chủ chuyên dụng (Dedicated Instances).⁷
- **VmSchedulerTimeShared (Phân bổ Thời gian Mở rộng):** Triển khai cơ chế chia sẻ ưu tiên (preemptive). Các vCPU của nhiều VM khác nhau có thể cùng chia sẻ một lõi PE vật lý. Nếu tổng lượng MIPS yêu cầu từ tất cả các VM vượt quá tổng MIPS phần cứng, bộ lập lịch sẽ chia nhỏ công suất theo tỷ lệ phần trăm phân đoạn thời gian (time-slicing). Hậu quả là các VM sẽ phải chia sẻ tài nguyên, dẫn đến kéo dài thời gian hoàn thành (Makespan) của các ứng dụng bên trong chúng. Cơ chế này mô phỏng môi trường máy ảo đa khách thuê (Multi-tenant IaaS) thông thường.⁷
- **Completely Fair Scheduler (CFS):** Một sự bổ sung độc quyền và phức tạp của CloudSim Plus, mô phỏng chính xác thuật toán CFS đang được sử dụng thực tế trong nhân Linux Kernel. Thuật toán này sử dụng cấu trúc cây Đỏ-Đen (Red-Black Tree) để theo dõi thời gian chạy ảo (vruntime) của từng VM và luân chuyển tài nguyên một cách công bằng nhất, mang lại độ chính xác cực cao cho các mô hình điện toán nhạy cảm với độ trễ.⁶

Mô hình Năng lượng và Tiêu thụ Điện năng

Một tính năng nổi bật khác trong việc mô phỏng Host là khả năng theo dõi điện năng (Power and Energy Awareness). Mỗi Host có thể được gán một đối tượng PowerModel (như tuyến tính, lập phương, hoặc mảng thông số rời rạc dựa trên SPEC benchmark) để liên tục dịch mức độ sử dụng CPU hiện tại sang lượng Watt tiêu thụ. CloudSim Plus cải tiến vượt bậc bằng cách theo dõi tự động điện năng ngay cả trong lúc máy chủ khởi động (startup), tắt (shutdown), cung cấp thông số về độ trễ bật/tắt (startup/shutdown delay) và lượng điện tiêu hao ở trạng thái nhàn rỗi (idle/sleep states).⁶

Vòng lặp Xử lý Động và Cơ chế Cập nhật Mô phỏng (Update Processing Loop)

Hệ thống đám mây là một khối kiến trúc chuyển động liên tục. Khi một khách hàng gửi các tác vụ (Cloudlets) lên các VM, hệ thống mô phỏng cần một cơ chế thông minh để tính toán xem khi nào các tác vụ này kết thúc, từ đó giải phóng tài nguyên. Cơ chế này không dùng luồng ngầm liên tục, mà được điều khiển bởi nhịp đập (heartbeat) định kỳ thông qua tham số `schedulingInterval` của Datacenter và chuỗi phương thức gọi đệ quy `updateVmsProcessing()`.³⁰

Chu kỳ Lập lịch (`schedulingInterval`), tính bằng giây, xác định độ phân giải thời gian mà Datacenter sẽ thức dậy để rà soát hệ thống. Quá trình xử lý diễn ra theo thứ tự phân tầng như sau:

1. Tại thời điểm được báo thức bởi lõi DES, Datacenter sẽ gọi phương thức `updateVmsProcessing(currentTime)` và lặp qua toàn bộ danh sách Host đang trực thuộc.³²
2. Mỗi Host nhận lệnh sẽ cập nhật trạng thái điện năng hiện hành, sau đó gọi tiếp lệnh cập nhật đối với tất cả các Vm mà nó đang chứa, cân đối lại lượng MIPS được cấp phát dựa trên thuật toán của VmScheduler.²⁶
3. Tại cấp độ Vm, đối tượng CloudletScheduler (cũng có các phiên bản chia sẻ thời gian và không gian tương tự Host) sẽ tính toán lượng hàng triệu chỉ lệnh (MI) mà mỗi Cloudlet đã thực thi được kể từ lần cập nhật gần nhất (dựa trên lượng MIPS thực tế VM nhận được nhân với khoảng thời gian đã trôi qua).⁷
4. Tổng số lượng MI mới được trừ vào tổng chiều dài (Length) của Cloudlet. Nếu độ dài còn lại chạm mức 0, Cloudlet được đánh dấu hoàn thành, SimEvent báo cáo kết quả được sinh ra và gửi về DatacenterBroker.
5. Sau khi cập nhật xong mọi thứ, các tầng dưới sẽ tính toán và trả về "thời gian dự kiến hoàn thành" (expected completion time) nhỏ nhất của các Cloudlet còn lại. Datacenter sẽ sử dụng mốc thời gian này để tự động tạo một SimEvent nội bộ, tự nhắc nhở hệ thống đánh thức nó vào đúng thời điểm đó trong tương lai để cập nhật lại.¹⁶

Độ dài của `schedulingInterval` mang tính thỏa hiệp sâu sắc giữa Độ chính xác (Accuracy) và Hiệu năng (Performance). Nếu thiết lập quá nhỏ (ví dụ 1 giây), các phương thức đệ quy này sẽ

được gọi hàng ngàn lần liên tục, hệ thống sẽ sinh ra hàng triệu sự kiện cập nhật nhét vào FutureQueue, gây sụt giảm hiệu suất nghiêm trọng. Nếu thiết lập quá dài, hệ thống chạy rất nhanh nhưng các tác vụ bị báo cáo hoàn thành trễ hơn thực tế, làm méo mó các số liệu về mức tiêu thụ năng lượng và phí thanh toán.³¹

Lắng nghe Sự kiện, Tự động Mở rộng và Phân bổ Khối lượng công việc

Điểm chấp nối mọi chức năng của CloudSim Plus thành một nền tảng tiên tiến là sự ra đời của Kiến trúc Lắng nghe Sự kiện (Event Listeners) thay vì cơ chế kế thừa (inheritance) cứng nhắc của CloudSim nguyên thủy.

Mạng lưới Listeners bằng Biểu thức Lambda

Sử dụng Functional Interfaces của Java 8+, CloudSim Plus cho phép nhà nghiên cứu truyền các đoạn mã vô danh (lambda expressions) vào các móc nối (hooks) của hệ thống thông qua mô hình Thiết kế Observer.¹ Các Listener phổ biến bao gồm:

- Kích hoạt mỗi khi đồng hồ mô phỏng tăng lên (addOnClockTickListener).³⁴
- Kích hoạt khi có một sự kiện được giải quyết (addOnEventProcessingListener).⁷
- Kích hoạt tại cấp độ DatacenterBroker khi phân bổ VM thất bại do thiếu Host vật lý (onCreationFailureListener). Dựa vào tín hiệu này, một Broker có thể kích hoạt chính sách dự phòng (fallback mechanism), thử cấp phát VM tại một Datacenter khác nằm ở vùng địa lý gần nhất.⁶

Tự động Mở rộng Quy mô Máy ảo (Auto-Scaling)

Sức mạnh của Listeners được minh chứng thông qua các cơ chế Độ co giãn Đám mây (Cloud Elasticity) được tích hợp sẵn. Hệ thống định kỳ sử dụng Listeners để lấy mẫu sự sử dụng CPU, RAM của VM và thực thi hai cơ chế mở rộng⁷:

1. **Mở rộng Chiều dọc (Vertical VM Scaling):** Nếu độ sử dụng CPU/RAM của VM chạm ngưỡng quá tải (Overload) hoặc dưới tải (Underload), cơ chế VerticalVmScaling sẽ được gọi. Nó tương tác trực tiếp với Hypervisor giả lập để tăng thêm hoặc giảm bớt trực tiếp MIPS, RAM, bằng thông cho VM đó trong khi đang chạy mà không cần khởi động lại. Điều này giúp ngăn chặn các cuộc di dời máy ảo (Live Migration) tiêu tốn năng lượng và thời gian.⁹
2. **Mở rộng Chiều ngang (Horizontal VM Scaling):** Trong trường hợp VM đã phình to tới giới hạn phần cứng của Host mà vẫn quá tải, HorizontalVmScaling sẽ ra lệnh cho hệ thống phá bản sao (cloning) cấu hình của VM đó, tự động gửi yêu cầu khởi tạo VM mới (scale-out) đến Broker để san sẻ tải trọng, mô phỏng chính xác tính năng Auto-Scaling Groups của AWS hoặc Google Cloud.⁷

Động lực Khối lượng Công việc (Workload Dynamics) và Lịch sử Dữ

liệu

Các ứng dụng đám mây hiếm khi chạy ở 100% CPU xuyên suốt thời gian. CloudSim Plus biểu diễn hành vi này thông qua các mô hình mức sử dụng (Utilization Models), ví dụ `UtilizationModelStochastic`, giúp sinh ra mức tiêu thụ tài nguyên ngẫu nhiên theo thời gian thực để tạo tải (stress) biến động lên CPU, RAM.⁴ Toàn bộ các thay đổi lịch sử về tài nguyên này đều được hệ thống âm thầm ghi chép lại vào đối tượng `HostStateHistoryEntry` hoặc lịch sử của VM ở từng mốc thời gian, tạo điều kiện thuận lợi cho các nghiên cứu dự đoán chuỗi thời gian (time-series forecasting) bằng Học máy (Machine Learning).⁶

Tích hợp Chuyên sâu: Spot Instances và Môi trường Liên kết Đám mây

Không dừng lại ở IaaS cơ bản, CloudSim Plus cung cấp nền tảng mở rộng để mô phỏng các mô hình tính giá phức tạp như Spot Instances (Máy ảo đấu giá). Khác với máy ảo theo yêu cầu (On-demand), Spot Instances mang lại chi phí rẻ nhưng có thể bị nhà cung cấp chấm dứt (preempted) đột ngột nếu năng lực trung tâm dữ liệu bị cạn kiệt do yêu cầu của người dùng ưu tiên cao hơn.¹²

Để mô phỏng hệ sinh thái này, CloudSim Plus đã được bổ sung một kiến trúc cho phép theo dõi toàn bộ vòng đời gián đoạn: từ khi nhận tín hiệu chấm dứt, trạng thái đóng băng tạm thời (hibernation), cho đến việc tự động tái đặt trình (dynamic reprovisioning) các tác vụ bị bỏ dở sang các Datacenter khác. Việc tích hợp các bộ thuật toán phân bổ máy ảo nâng cao như HLEM-VMP trực tiếp can thiệp vào cách hệ thống quản lý rủi ro trên thị trường Spot, tối ưu hóa việc phân tán các Cloudlet để giảm thiểu tối đa tổng thời gian gián đoạn (maximum interruption duration).¹²

Hơn nữa, thông qua cơ chế đọc dấu vết cụm (Trace-driven Simulation), hệ thống có khả năng nạp các bộ dữ liệu khổng lồ từ Google Cluster Trace. Thay vì định nghĩa tĩnh, hệ thống sẽ đọc từng luồng dữ liệu (JSON/CSV), tự động tạo các Host, mô phỏng sự kiện đến của các tác vụ (job arrivals) theo dòng thời gian chính xác của môi trường Google, liên tục đối chiếu với tài nguyên vật lý để điều chỉnh phân bổ.¹²

Đánh giá Tổng thể: Điểm mạnh, Điểm hạn chế (Pros & Cons)

CloudSim Plus hiện diện như một giải pháp thay thế mạnh mẽ, nhưng việc triển khai nó cho các mục tiêu nghiên cứu cụ thể đòi hỏi phải đánh giá thấu đáo các khía cạnh ưu/nhược điểm.

Điểm mạnh (Pros)

- **Tính Đúng đắn và Khả năng Tái lập (Accuracy and Reproducibility):** Kiến trúc xử lý sự kiện dựa trên Đối tượng đơn luồng triệt tiêu hoàn toàn lỗi không xác định (nondeterministic bugs) thường thấy trong lập trình đa luồng. Một kịch bản với cùng hạt

giống ngẫu nhiên (random seed) sẽ chạy ra kết quả chuẩn xác tới hàng thập phân ở bất kỳ máy tính nào.⁷

- **Thân thiện với Lập trình viên và An toàn Mã nguồn:** Việc xóa bỏ hàng ngàn dòng mã lặp lặp và triển khai các chuỗi gọi Object-Chaining (`cloudlet.getVm().getHost()`) giúp việc viết mã trở nên tinh gọn. Design Pattern Builder giúp khởi tạo các hệ thống chứa hàng chục thuộc tính chỉ trong một khối mã ngắn.¹
- **Mở rộng Đa miễn mạnh mẽ:** Hỗ trợ sẵn các cơ chế giả lập liên đám mây (Federated Clouds, Multi-cloud), mô phỏng mạng nội bộ theo topology phức tạp, xử lý đồng thời mô phỏng tiêu thụ điện năng kết hợp với tắc nghẽn băng thông.⁶
- **Dễ dàng Thu thập Dữ liệu (Metrics Collection):** Cung cấp các lớp TableBuilder hỗ trợ trích xuất thống kê trực tiếp sang định dạng CSV, HTML, Markdown giúp cho việc phân tích dữ liệu sau mô phỏng trở nên trực quan mà không cần viết mã thủ công rườm rà.⁶

Điểm hạn chế (Cons)

- **Thiếu vắng Giao diện Đồ họa (Lack of GUI):** Quản lý cấu hình bằng mã nguồn Java thuần túy hoặc cấu hình qua tệp YAML (CloudSim Plus Automation) tạo ra một rào cản lớn (Steep Learning Curve) cho những nhà nghiên cứu không chuyên sâu về khoa học máy tính hoặc lập trình hệ thống.⁶ Không có công cụ kéo thả để thiết lập các mô hình định tuyến trực quan.
- **Hạn chế Khả năng Tính toán Song song Lỗi (Single-threaded Core Loop Constraint):** Mặc dù khung làm việc cho phép thực thi *nhiều kịch bản mô phỏng độc lập* cùng lúc (Parallel Simulations) trên đa nhân CPU, vòng lặp xử lý của *từng môi trường riêng lẻ* hoàn toàn là đơn luồng.⁶ Quá trình tính toán sự kiện của một Datacenter không thể được phân tán ra nhiều lõi phần cứng (như các framework dựa trên MPI hoặc Actors), giới hạn mức thông lượng xử lý trần bực bởi xung nhịp đơn nhân (single-threaded performance) của CPU chạy máy ảo Java.³⁸

Giới hạn Quy mô và Các Điểm nghẽn khi Tăng lên Hàng vạn Host

Đối với các nghiên cứu đám mây quy mô vừa, CloudSim Plus duy trì phong độ xuất sắc. Khi so sánh trực tiếp tốc độ mô phỏng với phiên bản CloudSim 4.0.0 nguyên bản trên một cấu hình phần cứng tiêu chuẩn (CPU Intel i7 2 nhân 2.8 GHz, 8GB RAM) cho một kịch bản quy mô lớn gồm 1 Datacenter, **20.000 Host**, 40.000 VM và 50.000 Cloudlet, CloudSim Plus đã thể hiện sự vượt trội ⁴:

Thuật toán Phân bổ VM (VmAllocationPolicy)	Thời gian Mô phỏng CloudSim 4.0.0 (phút)	Thời gian Mô phỏng CloudSim Plus 4.3.x (phút)	Hiệu suất Cải thiện

Simple (WorstFit)	23.9 phút	15.7 phút	Nhanh hơn 34%
BestFit (Chỉ có ở CloudSim Plus)	19.6 phút	15.7 phút	Nhanh hơn 20%
FirstFit (Chỉ có ở CloudSim Plus)	13.6 phút	1.3 phút	Nhanh hơn >90%

Tuy nhiên, bảng số liệu này cũng vạch trần một thực tế: khi hệ thống cố gắng tăng tải vượt qua ngưỡng hàng chục ngàn Host hoặc mô phỏng với tập dữ liệu Google Cluster Trace kéo dài nhiều ngày, tài nguyên hệ thống sẽ chịu những sức ép khủng khiếp và phát sinh bốn điểm nghẽn (Bottlenecks) kiến trúc cốt lõi.

1. Điểm nghẽn Độ phức tạp Thuật toán Phân bổ (Allocation Matrix Complexity)

Bài toán gán VM vào Host trong CloudSim Plus thực chất là bài toán đóng thùng (Bin-packing). Khi một số lượng lớn VM được gửi đến Broker cùng lúc, Datacenter phải chạy thuật toán VmAllocationPolicy để lựa chọn Host phù hợp.¹²

- Đối với thuật toán **FirstFit**, chi phí tính toán là vô cùng nhỏ. Nó chỉ duyệt tuần tự danh sách Host, ngay khi tìm thấy Host đầu tiên thỏa mãn dung lượng tài nguyên (CPU, RAM, BW), nó sẽ gán và kết thúc vòng lặp. Đó là lý do kịch bản 20.000 Host chỉ mất 1.3 phút.⁴
- Tuy nhiên, đối với các thuật toán như **WorstFit** hoặc **BestFit**, hoặc các siêu heuristics, Datacenter buộc phải duyệt và chấm điểm (scoring) cho *toàn bộ* không gian Host để tìm

ra thiết bị tối ưu nhất.²² Gọi V là số lượng Virtual Machines cần phân bổ và H là số lượng máy chủ vật lý, độ phức tạp tính toán vọt lên mức $\mathcal{O}(V \times H)$.²² Nếu hệ thống gửi 10.000 VM vào một Datacenter chứa 10.000 Host, vòng lặp nội bộ bắt buộc phải thực hiện 100 triệu phép so sánh tài nguyên khắt khe. Nếu đẩy số Host lên 100.000, thời gian tính toán của CPU cho việc phân bổ sẽ leo thang theo cấp số nhân, khiến toàn bộ lõi mô phỏng bị kẹt cứng (blocked) trong hàng giờ.²²

2. Điểm nghẽn Cập nhật Độ quy (Recursive Update Overhead)

Như đã phân tích, hệ thống duy trì tính cập nhật của các tiến trình bằng phương thức `updateVmsProcessing()` được gọi định kỳ dựa trên `schedulingInterval`. Sự tàn phá tài nguyên của mô hình này lộ diện khi quy mô phình to. Nếu hệ thống có 20.000 Host, mỗi Host chứa 10 VM, và mỗi VM chứa 10 Cloudlet. Một lệnh cập nhật từ Datacenter sẽ kích hoạt một cây đệ quy duyệt qua $20.000 \times 10 \times 10 = 2.000.000$ đối tượng chỉ trong một nhịp đồng hồ.³⁰ Thử nghiệm mô phỏng cụm máy chủ thực tế với dữ liệu Google Cluster Trace (hơn 10.000 máy) đã báo cáo rằng: để chạy mô phỏng 2 ngày dữ liệu thực, hệ thống phải mất tới 7 ngày xử lý trên máy ảo Google Cloud có cấu hình 4 vCPU và 32GB RAM.¹² Đáng nói, kết quả giám sát chỉ ra rằng điểm nghẽn tính toán lớn nhất hoàn toàn không phải khâu phân bổ VM (allocation), mà chính là khối lượng công việc khổng lồ từ vòng lặp cập nhật tiến trình Cloudlet thực thi lặp đi lặp lại.¹²

3. Điểm nghẽn Hàng đợi Sự kiện Trung tâm (The Central Event Queue Choke)

Cấu trúc `FutureQueue` quản lý sự kiện dưới dạng cây nhị phân (Heap) để đảm bảo sự kiện ưu tiên (thời gian nhỏ nhất) luôn nổi lên đầu. Thao tác thêm một sự kiện mới hoặc lấy một sự kiện ra khỏi cấu trúc `Priority Queue` này có độ phức tạp thời gian là $O(\log N)$, trong đó N là tổng số sự kiện đang nằm trong hàng đợi.¹² Trong một cấu trúc mạng cực lớn mô phỏng tương tác vi dịch vụ (Microservices), định tuyến gói tin mạng (Network Packets) giữa các VM phân tán qua các Switch, hoặc mô phỏng với chu kỳ `schedulingInterval` ngắn, số lượng `SimEvent` chờ trong hàng đợi có thể nhảy vọt lên ngưỡng hàng chục triệu.¹² Khi N trở nên quá lớn, hệ số của $O(\log N)$ bắt đầu gây áp lực đáng kể lên hệ thống xử lý bộ đệm CPU (CPU Cache) và vòng lặp mô phỏng trung tâm, làm chậm tốc độ tổng thể thay vì các phép toán logic thực sự.¹³

4. Áp lực Quản lý Bộ nhớ và Thu gom Rác (JVM Garbage Collection)

Bộ nhớ Heap của máy ảo Java (JVM) tạo ra một điểm nghẽn hữu hình nhất về mặt phần cứng. Mặc dù `CloudSim Plus` đã tối ưu hóa cực đoan việc tái tạo định danh, thay thế việc nối chuỗi (String Concatenation) bằng `StringBuilder` để tránh tạo rác chuỗi, và ưu tiên kiểu dữ liệu nguyên thủy (primitive types) thay cho Wrapper classes như `Double`, `Integer` nhằm tối giản cấp phát đối tượng trên Heap⁵; tuy nhiên, quy mô khổng lồ của một thiết lập vạn Host sẽ tự tạo ra áp lực lưu trữ khủng khiếp. Một kịch bản dài ngày với 20.000 Hosts có thể sinh ra hàng triệu đối tượng `SimEvent`, lịch sử sử dụng tài nguyên (`HostStateHistoryEntry`), đối tượng mô phỏng bằng thông tính và hàng tấn log vận hành. Bộ nhớ Heap của JVM sẽ liên tục bị lấp đầy, kích hoạt cơ chế Thu gom Rác (Garbage Collection - GC) của Java.²³ Những chu kỳ quét rác lớn (Major GC Pauses) theo cơ chế G1GC hoặc Shenandoah sẽ buộc phải đóng băng toàn bộ tiến trình Java ("Stop-The-World"), làm đình trệ hoàn toàn sự tiến triển của mô phỏng. Đối với những dải mô phỏng khổng lồ trên hàng chục nghìn thực thể, việc sử dụng các công cụ cấu hình phần cứng (như `-Xmx32g` để nới rộng RAM lên 32GB) trở thành yêu cầu sống còn để tránh thảm họa treo

hệ thống (Out Of Memory Error).⁴¹

Tổng kết

CloudSim Plus đứng vững ở vị trí công cụ mô phỏng hàng đầu thế giới về điện toán đám mây, là cầu nối đặc lực để chứng minh các lý thuyết học thuật phức tạp. Thông qua việc đập bỏ thiết kế Đa luồng (Thread) nặng nề để áp dụng vòng lặp sự kiện Đối tượng (Object) đơn luồng, tích hợp các công cụ Lambda biểu thức cho các Listeners và hiện đại hóa cấu trúc máy chủ (Hosts) với các thuật toán phân chia tinh xảo như CFS hay Auto-Scaling; nền tảng này đã gia tăng sức mạnh mô phỏng lên nhiều lần. Tuy nhiên, kiến trúc xử lý sự kiện tập trung và tính toán cấp phát $O(N^2)$ vẫn tạo ra các rào cản vật lý ở quy mô siêu lớn. Việc thấu hiểu tường tận cấu trúc của FutureQueue, tác động của schedulingInterval, cùng khả năng kiểm soát Garbage Collector của máy ảo JVM là những kỹ năng không thể thiếu đối với các kỹ sư và nhà nghiên cứu muốn đẩy giới hạn thử nghiệm vượt qua ngưỡng chực vọt thiết bị điện toán.

Nguồn trích dẫn

1. CloudSim Plus: a Cloud Computing Simulation Framework Pursuing Software Engineering Principles for Improved Modularity, Extensibility and Correctness, truy cập vào tháng 5 15, 2026, <https://opendl.ifip-tc6.org/db/conf/im/im2017mini/047.pdf>
2. What is CloudSim? - GeeksforGeeks, truy cập vào tháng 5 15, 2026, <https://www.geeksforgeeks.org/cloud-computing/what-is-cloudsim/>
3. CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms | Request PDF - ResearchGate, truy cập vào tháng 5 15, 2026, https://www.researchgate.net/publication/220280566_CloudSim_A_Toolkit_for_Modeling_and_Simulation_of_Cloud_Computing_Environments_and_Evaluation_of_Resource_Provisioning_Algorithms
4. Performance Results | CloudSim Plus, truy cập vào tháng 5 15, 2026, <https://cloudsimplus.org/docs/performance.html>
5. CloudSim 7G: An Integrated Toolkit for Modeling and Simulation of Future Generation Cloud Computing Environments - arXiv, truy cập vào tháng 5 15, 2026, <https://arxiv.org/html/2408.13386v2>
6. CloudSim Plus | State-of-the-art Framework for Cloud Computing ☀️ Simulation: a modern, full-featured, easier-to-use, highly extensible , faster and more accurate ☕ Java 17+ tool for cloud computing research, truy cập vào tháng 5 15, 2026, <https://cloudsimplus.org/>
7. A White Paper about CloudSim Plus: A modern, full-featured, highly extensible and easier-to-use Java 8 Framework for Modeling and Simulation of Cloud Computing Infrastructures and Services ☀️ - GitHub, truy cập vào tháng 5 15, 2026, <https://github.com/cloudsimplus/cloudsimplus-whitepaper>
8. CloudSim Plus API 8.5.7 API - javadoc.io, truy cập vào tháng 5 15, 2026, <https://javadoc.io/doc/org.cloudsimplus/cloudsimplus>

9. CloudSim Plus: A Modern Java 8 Framework for Modeling and Simulation of Cloud Computing Infrastructures and Services, truy cập vào tháng 5 15, 2026, <https://cloudsimplus.github.io/cloudsimplus-whitepaper/>
10. tesis-cloudsimplus-v2 - Santiago Serantes Laforgue - GitLab de FING, truy cập vào tháng 5 15, 2026, <https://gitlab.fing.edu.uy/santiago.serantes/tesis-cloudsimplus-v2/-/blob/main/docs/cloudsim-plus-white-paper.pdf>
11. Passing IDs instead of entire objects is not an Object-Oriented approach · Issue #10 - GitHub, truy cập vào tháng 5 15, 2026, <https://github.com/cloudsimplus/cloudsimplus/issues/10>
12. Simulating Dynamic Cloud Marketspaces: Modeling Spot Instance Behavior and Scheduling with CloudSim Plus - arXiv, truy cập vào tháng 5 15, 2026, <https://arxiv.org/html/2511.18137v1>
13. Cloud and Web Services Overview | PDF - Scribd, truy cập vào tháng 5 15, 2026, <https://www.scribd.com/document/811136000/TYCS-SEM-VI-CLOUD-WS-FINAL>
14. Creating Virtual Machines in Cloud Lab | PDF - Scribd, truy cập vào tháng 5 15, 2026, <https://www.scribd.com/document/687282888/Cloud-Manual>
15. Creating a Simulation Framework for the Hadoop Distributed File System inside Heterogeneous Cloud Environments by using CloudSim - POLITesi - Politecnico di Milano, truy cập vào tháng 5 15, 2026, https://www.politesi.polimi.it/retrieve/91c7d12b-bcf7-4596-a2bb-f10327fb2cf9/20_12_Prizio.pdf
16. Modeling and Simulation of Scalable Cloud Computing Environments and the CloudSim Toolkit: Challenges and Opportunities - IC-Unicamp, truy cập vào tháng 5 15, 2026, https://www.ic.unicamp.br/~bit/ensino/mo809_1s13/papers/Cloud_Computing/Modeling%20and%20simulation%20of%20scalable%20Cloud%20computing%20environments%20and%20the%20CloudSim%20toolkit-%20Challenges%20and%20opportunities.pdf
17. A Survey on Cloud Computing Simulation and Modeling - ResearchGate, truy cập vào tháng 5 15, 2026, https://www.researchgate.net/publication/343431342_A_Survey_on_Cloud_Computing_Simulation_and_Modeling
18. Simulating Dynamic Cloud Marketspaces: Modeling Spot Instance Behavior and Scheduling with CloudSim Plus - arXiv, truy cập vào tháng 5 15, 2026, <https://arxiv.org/pdf/2511.18137>
19. Simulation (CloudSim Plus API 4.6.0 API) - javadoc.io, truy cập vào tháng 5 15, 2026, <https://www.javadoc.io/doc/org.cloudsimplus/cloudsim-plus/4.6.0/org/cloudbus/cloudsim/core/Simulation.html>
20. Cloud Computing Lab Practicals Guide | PDF | Virtual Machine - Scribd, truy cập vào tháng 5 15, 2026, <https://www.scribd.com/document/977904819/Cloud-Computing-Lab>
21. PerficientCloudSim: a tool to simulate large-scale computation in heterogeneous clouds, truy cập vào tháng 5 15, 2026,

- https://openresearch.surrey.ac.uk/view/pdfCoverPage?instCode=44SUR_INST&filePid=13140495990002346&download=true
22. Improving Cloud Task Scheduling in Cloud Sim Plus using Demand-Aware VM Selection, truy cập vào tháng 5 15, 2026, <https://sistemasi.ftik.unisi.ac.id/index.php/stmsi/article/download/6253/1223>
 23. CloudSim 7G: An Integrated Toolkit for Modeling and Simulation of Future Generation Cloud Computing Environments - arXiv, truy cập vào tháng 5 15, 2026, <https://arxiv.org/html/2408.13386v1>
 24. Implementing a synchronous simulation · Issue #205 - GitHub, truy cập vào tháng 5 15, 2026, <https://github.com/cloudsimplus/cloudsimplus/issues/205>
 25. Memory demand in Cloudsim, truy cập vào tháng 5 15, 2026, https://groups.google.com/g/cloudsim/c/I_GD1ls-Uel
 26. HostSimple (CloudSim Plus API 1.2.4 API) - javadoc.io, truy cập vào tháng 5 15, 2026, <https://www.javadoc.io/doc/org.cloudsimplus/cloudsim-plus/1.2.4/org/cloudbus/cloudsim/hosts/HostSimple.html>
 27. Comparing CloudSim and CloudSim Plus Java Simulation Scenarios, truy cập vào tháng 5 15, 2026, <https://cloudsimplus.org/docs/CloudSim-and-CloudSimPlus-Comparison.html>
 28. GitHub - cloudsimplus/cloudsim-cloudsimplus-allocation-issue: A simulation scenario showing the differences in the resource allocation policies inherited from CloudSim and their implementation in CloudSim Plus, truy cập vào tháng 5 15, 2026, <https://github.com/cloudsimplus/cloudsim-cloudsimplus-allocation-issue>
 29. A Systematic Parameter Analysis of Cloud Simulation Tools in Cloud Computing Environments - MDPI, truy cập vào tháng 5 15, 2026, <https://www.mdpi.com/2076-3417/13/15/8785>
 30. ContainerCloudSim: An environment for modeling and simulation of containers in cloud data centers - Unpaywall, truy cập vào tháng 5 15, 2026, <https://unpaywall.org/10.1002%2FSPE.2422>
 31. Datacenter (CloudSim Plus API 5.1.1 API) - javadoc.io, truy cập vào tháng 5 15, 2026, <https://javadoc.io/doc/org.cloudsimplus/cloudsim-plus/5.1.1/org/cloudbus/cloudsim/datacenters/Datacenter.html>
 32. (PDF) ContainerCloudSim: An environment for modeling and simulation of containers in cloud data centers - ResearchGate, truy cập vào tháng 5 15, 2026, https://www.researchgate.net/publication/304536577_ContainerCloudSim_An_environment_for_modeling_and_simulation_of_containers_in_cloud_data_centers_MODELING_AND_SIMULATION_OF_CONTAINERIZED_CLOUDS
 33. Some cloudlets don't finish executing - Google Groups, truy cập vào tháng 5 15, 2026, <https://groups.google.com/g/cloudsimplus/c/WhmdVmxuRUA>
 34. CloudSim (CloudSim Plus API 4.0.5 API) - javadoc.io, truy cập vào tháng 5 15, 2026, <https://javadoc.io/static/org.cloudsimplus/cloudsim-plus/4.0.5/org/cloudbus/cloudsim/core/CloudSim.html>

35. All Classes (CloudSim Plus API 5.1.1 API) - javadoc.io, truy cập vào tháng 5 15, 2026,
<https://javadoc.io/doc/org.cloudsimplus/cloudsim-plus/5.1.1/allclasses-index.html>
36. wyywyy23/cloudsim-plus: A modern, full-featured, highly extensible and easier-to-use Java 8+ Framework for Cloud Computing Simulation ☀️ - GitHub, truy cập vào tháng 5 15, 2026, <https://github.com/wyywyy23/cloudsim-plus>
37. STUDY OF SIMULATORS FOR CLOUD COMPUTING - Евразийский научный журнал, truy cập vào tháng 5 15, 2026,
<https://journalpro.ru/articles/study-of-simulators-for-cloud-computing/>
38. (PDF) Efficient Multi-Cluster Scheduling for Heterogeneous Workloads - ResearchGate, truy cập vào tháng 5 15, 2026,
https://www.researchgate.net/publication/397085910_Efficient_Multi-Cluster_Scheduling_for_Heterogeneous_Workloads
39. Efficient Multi-Cluster Scheduling for Heterogeneous Workloads - IEEE Xplore, truy cập vào tháng 5 15, 2026,
<https://ieeexplore.ieee.org/iel8/6287639/10820123/11222093.pdf>
40. Garbage Collection in Java: The Performance Benefits of Upgrading, truy cập vào tháng 5 15, 2026, <https://inside.java/2025/05/14/javaone-garbage-collection/>
41. diabahmed/drl-cloudsimplus-loadbalancer: This repository contains the code and resources for my Master's Thesis - GitHub, truy cập vào tháng 5 15, 2026,
<https://github.com/diabahmed/drl-cloudsimplus-loadbalancer>
42. GitHub - cloudsimplus/cloud-simulation-comparison: A shell script that creates the same simulation scenario in CloudSim and CloudSim Plus to enable comparing results and performance, truy cập vào tháng 5 15, 2026,
<https://github.com/cloudsimplus/cloud-simulation-comparison>